Encontré una mascota: reidentificación de perros por fotografía con ResNet50 y análisis discriminante lineal

Imer Vladimir Coaguila Patiño

Maestría en Ingeniería de Software, Machine Learning

Octubre de 2026

Resumen

Se presenta un microservicio que, a partir de la foto del rostro de un perro encontrado, busca en la galería de mascotas registradas, propone los tres candidatos más parecidos y, si el primero supera un umbral calibrado, muestra la última ubicación de su collar. Es un componente de la plataforma de geolocalización de mascotas *Pets Location*. Se usaron 4.168 fotografías reales de 564 perros de DogFaceNet, particionadas por identidad en entrenamiento, validación y prueba. Sobre las características de una ResNet50 preentrenada y congelada se compararon tres métodos con el mismo conjunto de prueba: la línea base sin entrenamiento, una cabeza siamesa con pérdida contrastiva y una proyección PCA + LDA. El método elegido en validación, PCA + LDA, alcanzó un Top-1 de 83,2 % frente a 73,2 % de la línea base, con un Recall@3 de 93,0 %. El sistema se entrega con API REST, interfaz web, 29 pruebas automatizadas y despliegue con Docker. También se reportan sus limitaciones: el umbral aceptó 8,8 % de perros no registrados en prueba frente al 5 % objetivo, y el sistema no detecta ni alinea rostros.

1 Problema y objetivo

Pets Location es la plataforma del proyecto de tesis del autor: registra mascotas, sus dueños y collares con GPS, y publica avisos de mascotas perdidas y encontradas. Cuando alguien encuentra un perro sin collar visible, no existe una forma rápida de saber si ya está registrado ni quién es su dueño. Este trabajo aborda esa necesidad con un componente de *Machine Learning*: identificar al perro a partir de una fotografía de su rostro.

La tarea se formula como búsqueda en galería de conjunto abierto. La unidad de observación es una fotografía del rostro de un perro. Dada una foto de consulta, el sistema

ordena los perros registrados por similitud y decide si el más cercano es suficientemente parecido para sugerirlo. Si no lo es, responde «sin coincidencia», porque el perro puede no estar registrado. Se prefiere a la verificación por pares («¿estas dos fotos son del mismo perro?») porque el uso real compara una foto contra todos los registrados.

Objetivo: implementar un flujo completo (datos, entrenamiento, evaluación, API y frontend) y comparar con honestidad el método propuesto frente a una línea base sencilla sobre el mismo conjunto de prueba. No se busca construir toda la plataforma, sino su funcionalidad inteligente, como un microservicio independiente que se ejecuta por sí solo.

2 Dataset y preparación

2.1 Origen

Se usó *DogFaceNet_224resized* (Mougeot et al., 2019), publicado en Zenodo (DOI 10.5281/zenodo.12578449) con licencia CC BY 4.0. Contiene 8.363 fotografías reales de 1.393 perros: caras ya recortadas y alineadas a 224×224 píxeles, organizadas en una carpeta por individuo. La anotación es la identidad del perro. El script de preparación verifica el checksum MD5 del archivo antes de usarlo. Los nombres de mascotas, dueños, collares y ubicaciones que muestra la aplicación son ficticios.

2.2 Limpieza, selección y partición

La preparación (`app/prepare_data.py`, semilla 42) aplica estos pasos:

1. Se descartan las identidades que comparten una foto idéntica o casi idéntica con otra identidad, por ser ruido de etiqueta (4 identidades). La comparación usa un *hash* perceptual de 64 bits.

2. Dentro de cada perro se eliminan duplicados exactos (SHA-256) y casi exactos (99 imágenes).

3. Se conservan los perros con al menos 6 fotos (564) y se toman como máximo 10 fotos por perro.

4. Se reparte por identidad en 60/20/20 %: ningún perro aparece en dos particiones, lo que evita la fuga entre registros relacionados.

5. En validación y prueba, el 75 % de los perros queda «registrado» (2 fotos a la galería y el resto como consultas) y el 25 % «sin registro» (todas sus fotos son consultas que el sistema debería rechazar).

Tabla 1: Partición del dataset por identidad.

Partición	Perros	Fotos	Uso
Entrenamiento	338	2.517	ajustar PCA + LDA y la cabeza siamesa
Validación	113 (85 + 28)	807	elegir hiperparámetros, método y umbral
Prueba	113 (85 + 28)	844	evaluación final, ejecutada una sola vez

En prueba, la galería tiene 170 fotos de 85 perros y hay 674 consultas: 459 de perros registrados y 215 de perros sin registro. El archivo `data/manifest.json` registra el origen, la partición y el rol de cada imagen, y una prueba automatizada verifica que no haya identidades ni imágenes compartidas entre particiones.

3 Modelo y línea base

Los tres métodos comparten el mismo extractor y solo difieren en cómo comparan sus vectores. Se distingue explícitamente lo reutilizado de lo entrenado.

3.1 Extractor reutilizado: ResNet50

Se usa ResNet50 (He et al., 2016) de `keras.applications` con pesos ImageNet (Deng et al., 2009), sin la capa de clasificación y con *global average pooling*. Cada foto produce un vector de 2.048 dimensiones. No se ajusta: está congelada en todos los métodos. Por eso sus características se calculan una sola vez y se guardan en caché (unos 7 minutos en CPU para todo el dataset), lo que hace que el entrenamiento posterior tarde segundos. La caché incluye además la versión volteada horizontalmente de las fotos de entrenamiento, única aumentación usada.

Las imágenes se preprocesan igual en entrenamiento, evaluación y servicio: corrección de orientación EXIF, conversión a RGB, recorte central cuadrado, redimensión a 224×224 y normalización de ResNet50.

3.2 Línea base

Los vectores de ResNet50 se normalizan a norma 1 y se comparan por distancia euclídea. Sobre vectores unitarios, $d = \sqrt{2 - 2\cos\theta}$, por lo que el orden coincide con el de la distancia coseno. No se entrena nada: solo se calibra el umbral en validación.

3.3 Método propuesto: PCA + LDA

El análisis discriminante lineal (Fisher, 1936; Belhumeur et al., 1997) busca la proyección W que maximiza la separación entre individuos respecto de la variación de un mismo individuo:

$$W^* = \arg\max_W \frac{\left|W^\top S_B W\right|}{\left|W^\top S_W W\right|}, \tag{1}$$

donde S_B es la dispersión entre clases (perros) y S_W la dispersión dentro de cada clase. Con 2.048 dimensiones y unas 5.000 muestras de entrenamiento (incluidas las volteadas), S_W está mal condicionada. Por eso primero se reduce a 512 dimensiones con PCA y después se usa LDA con *shrinkage* automático de Ledoit-Wolf (Ledoit y Wolf, 2004). El pipeline completo es: normalización L2 → PCA(512) → LDA(128) → normalización L2. Todo el pipeline es lineal, así que se guarda como una única transformación afín (W, b) de 2.048 a 128 dimensiones, aplicable con NumPy. Se implementó con scikit-learn 1.5.2 (Pedregosa et al., 2011).

3.4 Comparación adicional: red siamesa

Como alternativa de aprendizaje profundo se incluyó una red siamesa, el enfoque habitual de aprendizaje métrico para decidir si dos imágenes corresponden al mismo individuo (Hadsell et al., 2006). Las dos fotos de un par pasan por la misma cabeza, con pesos compartidos, y el entrenamiento acerca los vectores de un mismo perro y aleja los de perros distintos. Se consideró para contrastar el método clásico con uno aprendido por descenso de gradiente sobre las mismas características. La cabeza es una capa densa seguida de normalización L2, entrenada en TensorFlow 2.18 (Abadi et al., 2016) con pérdida contrastiva sobre pares positivos y negativos:

$$L = y\, D^2 + (1 - y)\, \max(0,\, m - D)^2, \tag{2}$$

donde D es la distancia entre los dos vectores, $y = 1$ si las fotos son del mismo perro y m es el margen. En cada época se generan de nuevo 5.034 pares balanceados. Se usa Adam con tasa 10^{-4}, lotes de 64 y como máximo 30 épocas, con detención temprana por el Top-1 de validación (paciencia 8).

3.5 Selección de hiperparámetros

Toda la selección usa solo validación. En la cabeza siamesa se probaron 4 configuraciones (dimensión 128 o 256, activación ReLU o lineal, margen 0,5); en PCA + LDA, 6 (64, 128 o 256 componentes, con o sin *shrinkage*). Antes se exploraron en validación otras variantes de la cabeza (más dimensiones, *dropout*, otras tasas de aprendizaje) sin mejoras consistentes. El método por defecto del servicio se fija con la regla «mayor Top-1 en validación» antes de

evaluar la prueba.

Tabla 2: Selección en validación (Top-1 sobre 449 consultas de perros registrados).

Método	Configuración	Top-1 val.
Línea base	sin entrenamiento	81,5 %
Cabeza siamesa	128, ReLU (configuración inicial)	79,1 %
Cabeza siamesa	256, lineal (mejor, época 3)	85,3 %
PCA + LDA	128 componentes, sin *shrinkage*	87,1 %
PCA + LDA	128 componentes, *shrinkage* automático (**elegido**)	**87,8 %**

La configuración inicial de la cabeza siamesa quedó por debajo de la línea base en validación. Todas las variantes alcanzaron su mejor punto en las primeras épocas y luego empeoraron: con 338 perros, aprender la proyección por descenso de gradiente sobre pares sobreajusta rápido. LDA, en cambio, obtiene su proyección en forma cerrada a partir de todas las muestras por clase, lo que explica su mejor comportamiento con pocos datos.

4 Métricas y validación

4.1 Métricas

Para cada consulta, la distancia a un perro registrado es la mínima a sus fotos de galería. Se reportan:

- Top-1 (métrica principal): proporción de consultas de perros registrados cuyo primer candidato es el perro correcto. Es lo que la interfaz destaca al usuario.
- Recall@3: el perro correcto está entre los tres candidatos mostrados.
- Tasa de identificación con umbral: consultas de registrados aceptadas y con la identidad correcta. Es el comportamiento real del sistema.
- FAR de desconocidos: proporción de consultas de perros sin registro que el sistema acepta por error. Es el error más costoso: sugerir un dueño equivocado.
- ROC-AUC y EER de verificación: sobre todos los pares (consulta, perro registrado), independientes del umbral, que miden qué tan bien se separan los pares del mismo perro de los pares de perros distintos.

4.2 Protocolo de validación

El umbral de cada método se calibra en validación: es el valor que maximiza la tasa de identificación con un FAR de desconocidos de 5 % como máximo. La prueba se ejecutó una sola vez, al final, con los umbrales y el método ya fijados, y ningún parámetro se ajustó después con sus resultados. Como la partición es por identidad, los 113 perros de prueba no fueron vistos en entrenamiento ni en validación.

5 Arquitectura e interfaz

El sistema es un microservicio independiente de la tesis, ejecutable con `python run.py` o `docker compose up` en el puerto 9100 (Figura 1). La parte *offline* produce los artefactos: el manifiesto de datos, la transformación PCA + LDA, los pesos de la cabeza, la calibración, la galería y las métricas. La parte *online* los carga al iniciar, ejecuta una pasada de calentamiento del modelo y atiende las solicitudes.

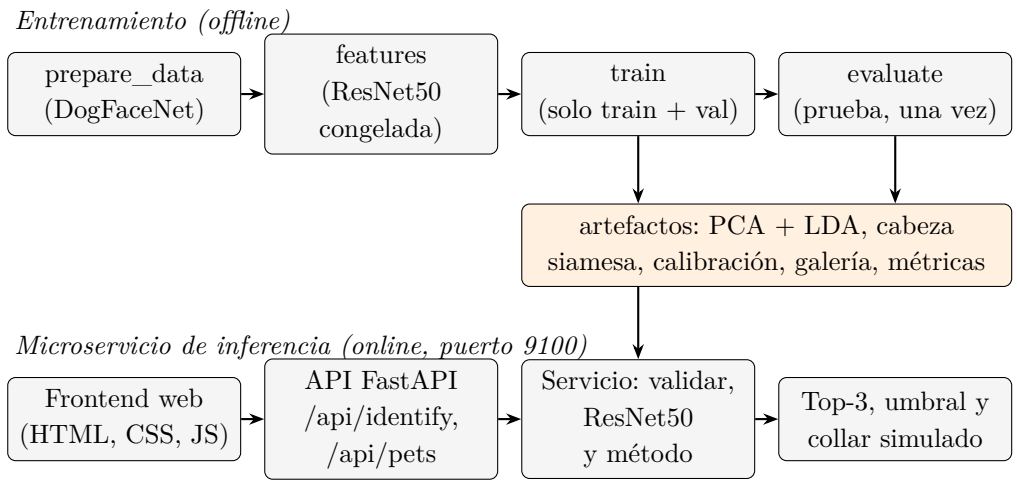


Figura 1: Arquitectura: flujo de entrenamiento *offline* y microservicio de inferencia.

Tabla 3: Endpoints principales de la API.

Método	Ruta	Función
POST	`/api/identify`	imagen y método opcional → top-3, decisión y ubicación
POST	`/api/pets`	registra una mascota con 2 a 4 fotos
POST	`/api/pets/{id}/ping`	simula una nueva señal del collar
GET	`/api/model`	métodos, umbrales y métricas
GET	`/api/gallery`	galería con la última ubicación de cada collar
GET	`/api/health`	estado del servicio

La API valida cada imagen antes de usar el modelo. Responde 415 si el archivo no es PNG o JPEG (se verifica el contenido, no la extensión), 413 si supera los 5 MB y 422 si mide menos de 96 píxeles por lado, supera los 6 megapíxeles, está dañada o el método no existe. La ubicación del collar se simula con el mismo modelo de datos (`Device`, `Location`) y la misma zona de Cochabamba que la tesis.

La interfaz (Figura 2) permite elegir una foto de ejemplo del conjunto de prueba o subir una propia, seleccionar el método y ver los tres candidatos con su distancia frente al umbral. Si hay coincidencia, muestra el dueño y la última ubicación del collar. Incluye estados de carga, mensajes de error (Figura 4) y un aviso permanente con los límites de la solución. Las seis fotos de ejemplo son fijas: dos perros registrados, dos no registrados y dos errores conocidos del modelo, para que la demostración sea reproducible. También permite registrar una mascota nueva y buscarla después con otra foto.

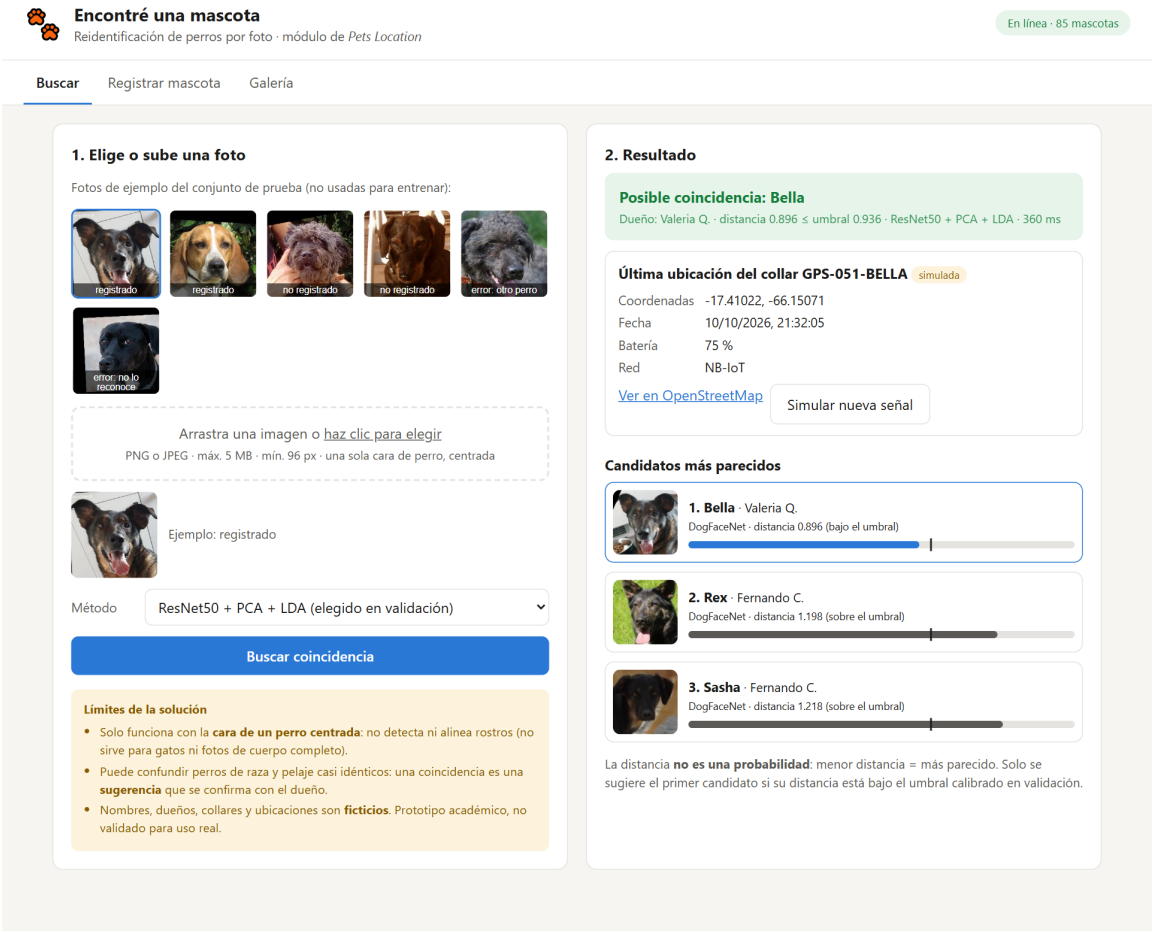


Figura 2: Caso válido: un perro registrado del conjunto de prueba se identifica (distancia 0,896 $\leq$ umbral 0,936) y se muestra la ubicación simulada de su collar.



Figura 3: Caso difícil: un perro sin registro se rechaza (0,969 $>$ 0,936).



Figura 4: Entrada inválida: un archivo de texto con extensión `.jpg` se rechaza con un mensaje claro.

6 Resultados

La Tabla 4 resume la evaluación en prueba. Los tres métodos usan las mismas 674 consultas y la misma galería.

Tabla 4: Resultados en prueba (113 perros nunca vistos). El umbral de cada método proviene de validación.

Métrica	Línea base	Siamesa	**PCA + LDA**
Top-1 (principal)	73,2 %	76,0 %	**83,2 %**
Recall@3	86,5 %	90,4 %	**93,0 %**
Tasa de identificación con umbral	42,3 %	49,9 %	**55,3 %**
FAR de desconocidos	**5,6 %**	6,5 %	8,8 %
Registrados aceptados con otro perro	**6**	12	14
Registrados rechazados	259	218	**191**
ROC-AUC (verificación)	0,969	0,987	**0,992**
EER (verificación)	8,5 %	6,3 %	**4,7 %**
Umbral (de validación)	0,636	0,197	0,936

PCA + LDA mejora el Top-1 en 10 puntos sobre la línea base y en 7,2 sobre la cabeza siamesa. El orden entre métodos es el mismo que en validación, lo que indica que la selección no fue un efecto del azar de esa partición. La curva Recall@k (Figura 5) muestra que la ventaja se mantiene para todo k, y la curva ROC (Figura 6) confirma la mejora en verificación.

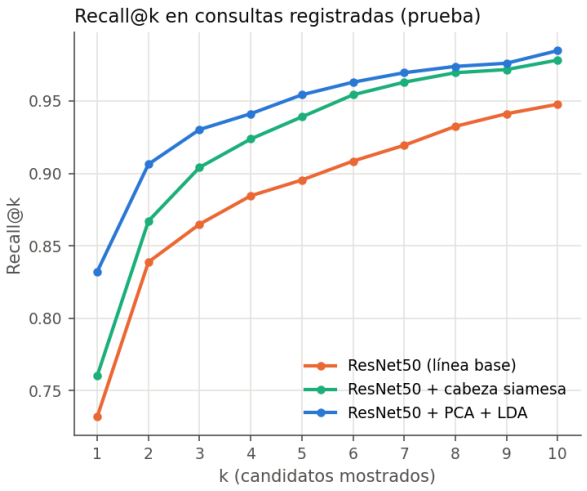


Figura 5: Recall@k en consultas de perros registrados.

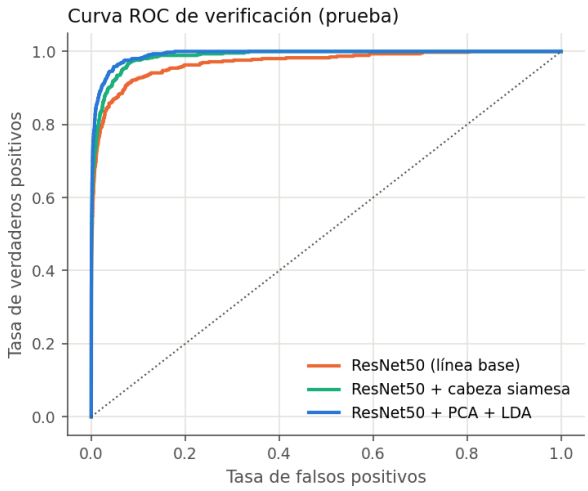


Figura 6: Curva ROC de verificación en prueba.

6.1 Análisis de errores

El umbral no se transfiere perfectamente. Los tres umbrales se calibraron para aceptar como máximo 5 % de desconocidos en validación. En prueba, PCA + LDA aceptó 8,8 % (19 de 215 consultas). La Figura 7 muestra el motivo: las distribuciones de distancias de perros registrados y no registrados se solapan, y un pequeño desplazamiento entre particiones cambia la proporción aceptada. Se reporta tal cual: corregirlo con la prueba invalidaría la evaluación.

El umbral es conservador. Aunque el perro correcto sale primero en el 83,2 % de los casos, solo el 55,3 % se acepta: muchos aciertos quedan sobre el umbral. Es el costo de limitar las falsas aceptaciones, y la razón por la que la interfaz muestra siempre los tres candidatos.

Perros casi idénticos. En el 16,8 % de las consultas de perros registrados el primer candidato no es el correcto, incluso con PCA + LDA. Entre los errores aceptados con la identidad equivocada (Figura 8) predominan perros de pelaje y raza casi idénticos, por ejemplo negro con fuego, y fotos rotadas por la alineación del dataset.

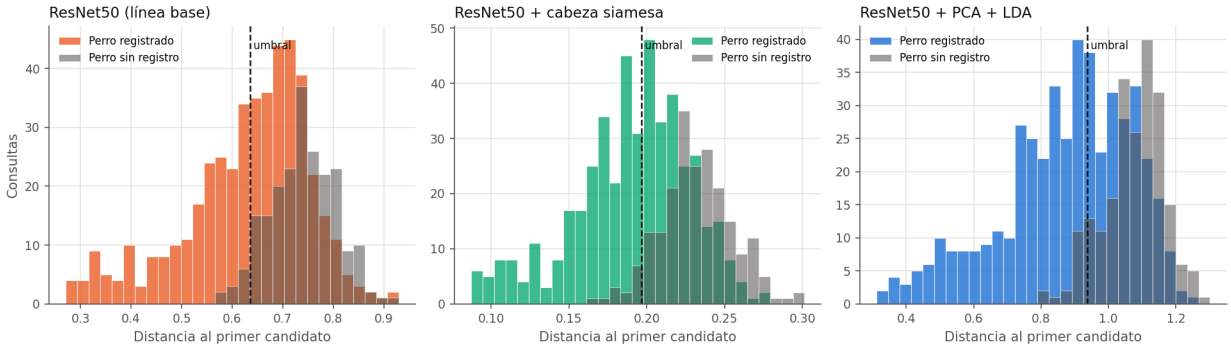


Figura 7: Distancia al primer candidato en prueba para perros registrados y sin registro; la línea discontinua es el umbral de validación.



Figura 8: Errores de PCA + LDA aceptados con la identidad equivocada: consulta (izquierda) y foto registrada del perro sugerido (derecha).

7 Pruebas realizadas

El proyecto incluye 29 pruebas automatizadas con `pytest`. Las pruebas de la API usan el modelo real, no respuestas fijas:

- Caso válido: una foto de prueba de un perro registrado se identifica con la identidad correcta y su ubicación. Además, para los tres métodos, el top-1 y la distancia de la API coinciden con los de la evaluación *offline*.
- Casos difíciles: un perro sin registro (la decisión de la API coincide con la evaluación), un perro registrado que el modelo confunde (la API reproduce el mismo error documentado) y una imagen sin perro, que se rechaza por el umbral.
- Entradas inválidas: texto con extensión `.jpg` (415), imagen de 40 px (422), archivo de más de 5 MB (413), método inexistente (422), solicitud sin archivo (422) y registro con una sola foto (422).

- Flujo completo: registrar un perro no registrado con 2 fotos, identificarlo con una tercera, simular una señal del collar y eliminarlo.
- Protocolo y preprocesamiento: no hay identidades ni imágenes compartidas entre particiones, los roles de galería y consulta son correctos, se aplica la orientación EXIF y se convierten las imágenes con transparencia.
- Demostración preparada: las seis fotos de ejemplo de la interfaz mantienen el resultado que indica su etiqueta (registrado, no registrado y los dos errores conocidos).

La ejecución siguiendo el README se verificó en una copia limpia del repositorio, con un entorno virtual nuevo: todas las pruebas pasaron y el servidor respondió. También se verificó la construcción y ejecución con Docker. Tras el calentamiento inicial, una identificación tarda entre 0,6 y 1 s en CPU.

8 Conclusiones y limitaciones

8.1 Conclusiones

Se implementó un sistema funcional de extremo a extremo: preparación reproducible de datos reales, entrenamiento, evaluación honesta, API, interfaz y pruebas. El resultado principal es que un método clásico de *Machine Learning*, LDA sobre características de una red preentrenada, supera a la línea base en 10 puntos de Top-1 (83,2 % frente a 73,2 %) y también a la red siamesa. Con pocos individuos de entrenamiento, una proyección discriminante en forma cerrada aprovecha mejor los datos que el aprendizaje por pares con descenso de gradiente. La separación por identidad, la calibración solo en validación y la evaluación única de la prueba hacen que la comparación sea confiable.

8.2 Limitaciones y trabajo futuro

El sistema necesita la cara del perro centrada: no detecta ni alinea rostros, y una foto de celular de cuerpo completo cambia de dominio respecto de las caras alineadas de entrenamiento. El umbral aceptó más desconocidos de lo previsto (8,8 % frente a 5 %), por lo que una coincidencia debe tratarse como sugerencia y confirmarse con el dueño. El dataset es pequeño para aprendizaje métrico (564 perros, de 6 a 10 fotos cada uno), y la galería y las ubicaciones de la demo son simuladas. Como trabajo futuro se proponen: un detector y alineador de rostros, la calibración del umbral con más perros no registrados, el ajuste fino de los últimos bloques de ResNet50 con más datos y la integración real con el backend Spring Boot de *Pets Location*.

Repositorio: `https://github.com/imertetsu/pet-reid-service` (versión final: tag `v1.0`).

Glosario

Vector de características (*embedding*). Lista de números que describe una imagen; fotos parecidas producen vectores cercanos. ResNet50 genera 2.048 números por foto.

Transfer learning. Reutilizar un modelo ya entrenado con otros datos (aquí ResNet50 con ImageNet) en lugar de entrenarlo desde cero.

Distancia. Medida de diferencia entre dos vectores: pequeña indica fotos parecidas, grande indica fotos distintas. No es una probabilidad.

Línea base. Método sencillo, sin entrenamiento, usado como punto de comparación para saber si el modelo propuesto aporta una mejora real.

PCA (Análisis de Componentes Principales). Técnica no supervisada que resume muchas variables en menos, conservando las direcciones de mayor variación. No usa las etiquetas.

LDA (Análisis Discriminante Lineal). Técnica supervisada que usa las etiquetas para hallar la proyección que agrupa las fotos de un mismo perro y separa las de perros distintos.

Shrinkage. Ajuste que estabiliza la estimación de LDA cuando hay pocas muestras para muchas variables.

Red siamesa y pérdida contrastiva. Modelo que procesa dos fotos con los mismos pesos y aprende a acercar los pares del mismo perro y alejar los de perros distintos.

Entrenamiento, validación y prueba. Particiones para aprender, para elegir método y umbral, y para la evaluación final única, respectivamente.

Fuga de datos. Error de protocolo en el que información de la prueba influye en el entrenamiento, por ejemplo el mismo perro en dos particiones; produce resultados optimistas.

Sobreajuste. El modelo memoriza los datos de entrenamiento y generaliza mal a datos nuevos.

Umbral. Distancia máxima a la que el sistema acepta al primer candidato como coincidencia.

Top-1. Proporción de consultas en las que el perro correcto aparece en primer lugar.

Recall@k. Proporción de consultas en las que el perro correcto está entre los k primeros candidatos.

Tasa de identificación. Proporción de consultas de perros registrados que el sistema acepta con la identidad correcta.

FAR (tasa de falsa aceptación). Proporción de perros no registrados a los que el sistema sugiere un dueño por error.

Curva ROC. Gráfico de la tasa de verdaderos positivos frente a la de falsos positivos al recorrer todos los umbrales posibles.

AUC. Área bajo la curva ROC: 1 indica separación perfecta y 0,5 equivale al azar.

EER (tasa de error igual). Punto en el que la tasa de falsas aceptaciones iguala a la de falsos rechazos; menor es mejor.

Referencias

- Belhumeur, P., Hespanha, J. y Kriegman, D. (1997). Eigenfaces vs. Fisherfaces: Recognition using class specific linear projection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(7), 711–720.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. y Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *CVPR 2009*.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
- Hadsell, R., Chopra, S. y LeCun, Y. (2006). Dimensionality reduction by learning an invariant mapping. *CVPR 2006*.
- He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep residual learning for image recognition. *CVPR 2016*.
- Ledoit, O. y Wolf, M. (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2), 365–411.
- Mougeot, G., Li, D. y Jia, S. (2019). A deep learning approach for dog face verification and recognition. *PRICAI 2019*, LNCS 11672. Dataset: https://doi.org/10.5281/zenodo.12578449
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Abadi, M. et al. (2016). TensorFlow: A system for large-scale machine learning. *OSDI 2016*.